

ULK POLYTECHNIC INSTITUTE

Assessment Technical Report: Cryptography and Network Security
Module: ETTCS801 Cryptography and Network Security — Assessment: Integrated Situation

Student ID: 4202670023
Lecturer: Isaac TUMWINE

September 28, 2026

1 Introduction

A security assessment was conducted for ULK Polytechnic Institute following identified vulnerabilities, including weak staff passwords, outdated software, unencrypted inter-campus file transfers, guest network exposure to the central student records server, and suspicious external access attempts. This report outlines the risk assessment findings, python-based cryptographic controls implemented, network traffic filtering rules configured, and verification test results.

2 Risk Assessment and Recommended Controls

Three critical risks were identified, evaluated, and prioritized based on their likelihood and impact on organizational assets.

Asset		Vulnerability	Consequence	Risk Level
Central Server	Records	Direct guest network exposure	Unauthorized internal access and data tampering	High
Inter-campus Transfers	Data	Unencrypted file transfers & weak staff credentials	Interception via packet sniffing & credential compromise	High
External Interface		Unrestricted external access probes	Exploitation of outdated software via automated scans	Medium

Table 1: Risk Assessment Summary Table

2.1 Recommended Security Controls

- Subnet Isolation & Traffic Filtering:** Enforce strict packet filtering ('iptables') to block the guest subnet from reaching the records server.
- Cryptographic Protection:** Utilize symmetric encryption (AES via Fernet) for data at rest and in transit, decoupled from source code using environment variables.
- Access Control & Policy Enforcement:** Mandate strong password policies and restrict management services (SSH) strictly to authorized staff IP ranges.

3 Encryption, Decryption, and Integrity Implementation

A custom Python security toolkit (`'security_toolkit.py'`) was developed to secure student records.

3.1 Cryptographic Mechanism

The tool leverages the `'cryptography.fernet'` library, implementing standard AES-128 in Cipher Block Chaining (CBC) mode with PKCS7 padding, combined with HMAC-SHA256 authentication. To maintain security best practices, encryption keys are maintained strictly outside source control and retrieved dynamically via the `'APP_ENCRYPTION_KEY'` environment variable.

3.2 Integrity Verification & Error Handling

File integrity is validated using SHA-256 cryptographic hashing before encryption and following decryption. The application includes input validation and exception handling (`'FileNotFoundError'`, `'InvalidToken'`) ensuring that missing files, invalid inputs, or modified ciphertexts are detected gracefully without crashing execution.

3.3 Test Execution and Encryption Results

The script was executed using a sample student record dataset (`'sample_students.csv'`). The cryptographic toolkit

Listing 1: Cryptographic Toolkit Execution Output Log

```
[*] Environment variable 'APP_ENCRYPTION_KEY' not found. Generating
    temporary session key...
[+] Created sample input file: 'sample_students.csv'

--- Step 1: Encrypting File ---
[+] File successfully encrypted: 'sample_students.csv.enc'

--- Step 2: Decrypting File ---
[+] File successfully decrypted: 'sample_students_decrypted.csv'

--- Step 3: Verifying File Integrity ---
Original File SHA-256 : 2
    a38df4cdd406cfc81e6af3d1a691f91e08697ce497e1e21ed633007105f02ee
Decrypted File SHA-256 : 2
    a38df4cdd406cfc81e6af3d1a691f91e08697ce497e1e21ed633007105f02ee
[SUCCESS] Integrity Match: The file has not been altered.

--- Step 4: Robustness & Error Testing ---
[!] Decryption Error: Encrypted file 'non_existent_file.enc' not found.
```

As evidenced by the execution logs, the original and decrypted SHA-256 hashes matched identically (2a38df4c...), confirming zero bit corruption or payload modification. Furthermore, when provided with a missing file input, the application handled the error gracefully without crashing.

4 Network Traffic Filtering and Test Evidence

Traffic filtering rules were configured using `'iptables'` on the target records server (`'192.168.10.100'`) to isolate management services (SSH Port 22).

4.1 Firewall Rule Definitions

Listing 2: Applied IPTables Rules

```
# Drop all guest network access to records server
sudo iptables -A INPUT -s 192.168.20.0/24 -d 192.168.10.100 -j DROP

# Permit authorized staff network access to SSH
sudo iptables -A INPUT -p tcp -s 192.168.10.0/24 --dport 22 -d
192.168.10.100 -j ACCEPT

# Block all other inbound access to SSH
sudo iptables -A INPUT -p tcp --dport 22 -j DROP
```

4.2 Test Procedures and Results

Network connectivity was verified using ‘netcat’ (‘nc’) across three scenarios:

1. **Permitted Staff Connection:** Initiated from ‘192.168.10.15’ → **Result:** Connection succeeded.
2. **Blocked Guest Connection:** Initiated from ‘192.168.20.50’ → **Result:** Connection timed out (Dropped).
3. **Blocked External Connection:** Initiated from ‘198.51.100.44’ → **Result:** Connection timed out (Dropped).

5 Conclusion

The security controls successfully mitigate the key risks identified during the security review. Sensitive student data is protected from unauthorized network exposure via firewall rules, and its confidentiality and integrity are ensured through robust cryptographic routines.

GitHub Repository URL: <https://github.com/olivekayitesi16-jpg/cryptography-network-security>